



MODERN AI ENGINEERING

Setting Up Your AI Dev Environment, the **Right Way**

venv, pip, uv, API keys, .env files, and every config file you actually need.



AI Coach John



FOLLOW

Repost



FOUNDATION & BENEFITS

A messy setup causes 90% of "why isn't this working" bugs

Before writing a single line of AI code, a clean environment saves you from:

- ✗ Package version conflicts between projects
- ✗ Accidentally leaking API keys onto GitHub
- ✗ "Works on my machine" problems when sharing code
- ✗ Hours lost debugging something that was actually a missing dependency

☰ What we will set up, step by step:

1

An isolated virtual environment ([venv](#) or [uv](#))

2

Dependency management ([pip](#) or [uv](#))

3

Secure API key storage ([.env](#) files)

4

A proper project folder structure

5

Config files that make your project portable, shareable, and tunable



AI Coach John



FOLLOW

Repost



Virtual Environments: **venv**

Why every project needs its own isolated environment

What it does: Creates an isolated Python installation just for this project, so its dependencies do not clash with other projects (or your system Python).

bash - venv setup

```
# Create a virtual environment named "venv"
python -m venv venv

# Activate it
source venv/bin/activate      # macOS/Linux
venv\Scripts\activate        # Windows

# Confirm you're in it (should show path inside project)
which python

# Install packages the standard way
pip install transformers langchain

# Deactivate when done
deactivate
```



venv ships with Python, requiring no extra install, which is why it is the default starting point for beginners. Always activate before running `pip install`.



Virtual Environments & Packages: **uv**

What most teams are switching to in 2026

What it does: `uv` (by Astral) is a single Rust-based tool that replaces `venv`, `pip`, `pip-tools`, and `pyenv`, often 10-100x faster with a built-in lockfile for reproducible installs.

bash - uv workflow

```
# Install uv (one-time, on your machine)
curl -LsSf https://astral.sh/uv/install.sh | sh
# or: pip install uv

# Start a new project, scaffolds pyproject.toml automatically
uv init my-ai-project
cd my-ai-project

# Add packages: creates .venv AND installs in one step
uv add transformers langchain langgraph python-dotenv

# Run code inside project env (no manual activation needed)
uv run main.py
```



Core Advantage: `uv add` auto-creates `.venv`, updates `pyproject.toml`, and writes exact versions to `uv.lock`. No manual venv activation required when using `uv run`.



AI Coach John



FOLLOW

 **Repost**



venv + pip vs uv: Quick Comparison

Which should you actually use?

FEATURE / METRIC	VENV + PIP	UV (MODERN)
Install speed	Standard	10-100x faster
Built-in with Python	Yes	No (one-time install)
Dependency resolution	Basic	Much stronger & cleaner
Lockfile support	Needs pip-tools	Built-in (uv.lock)
Manual venv activation?	Yes	No (with uv run / add)
Beginner-friendliness	Simplest to learn first	Slightly more upfront
Industry adoption	Still extremely common	Growing fast, default choice



Recommendation: Learn `venv` + `pip` first to understand what is happening under the hood, then switch to `uv` once comfortable since it is largely a faster, automated superset.





pip Basics: Installing & Freezing

Managing what your project depends on

What it does: `pip` installs packages; `requirements.txt` records exactly which packages (and versions) your project needs so others can recreate your environment.

bash - pip commands

```
# Install a package with pinned version
pip install transformers==4.44.0

# Install everything from a requirements file
pip install -r requirements.txt

# Save your current environment's packages to a file
pip freeze > requirements.txt
```

`requirements.txt` example

```
transformers==4.44.0
langchain==0.2.11
faiss-cpu==1.8.0
python-dotenv==1.0.1
```



Best Practice: Pin exact versions (`==`) for packages you actively use. Unpinned dependencies can silently break your code when libraries update.



uv's Two Modes: Project vs pip-Compatible

These are different commands; know which to use

uv offers two distinct workflows. Do not mix them up inside the same project:

```
bash - two modes of uv

# MODE 1: Project workflow (recommended for new projects)
# Tracks in pyproject.toml + uv.lock automatically
uv add transformers langchain
uv add --dev pytest black           # dev-only dependencies
uv remove langchain                 # removes + updates lockfile
uv sync                             # installs exact uv.lock items

# MODE 2: pip-compatible mode (drop-in pip replacement)
# Does NOT update pyproject.toml: manual, closer to plain pip
uv pip install transformers
uv pip install -r requirements.txt
uv pip freeze > requirements.txt
```



Use **uv add** / **uv sync** (Mode 1) for new projects to gain automatic lockfile tracking (**uv.lock**), ensuring identical builds across all machines.



AI Coach John



FOLLOW

 Repost

API Keys: What They Are & Why They're Dangerous

The #1 beginner security mistake

An API key is a secret credential authenticating your requests to services (OpenAI, Anthropic, HuggingFace) and is tied directly to your billing.

Never do this in your code:

```
# NEVER hardcode keys directly in your source files
client = OpenAI(api_key="sk-abc123...") # exposed if pushed!
```

Why it matters:

- Committed keys are scraped by bots within minutes of hitting GitHub.
- A leaked key can rack up massive billing charges before you notice.
- Keys can leak via chat copy-paste, screenshots, or repo visibility shifts.





SECRET STORAGE PATTERN

.env Files: Storing Secrets Safely

The standard way to keep API keys out of your code

What it does: Stores secrets in a separate file loaded into environment variables at runtime, which is never committed to version control.



.env file (in project root)

```
OPENAI_API_KEY=sk-abc123...  
ANTHROPIC_API_KEY=sk-ant-...  
HUGGINGFACE_TOKEN=hf_...
```

Python code using python-dotenv

```
from dotenv import load_dotenv  
import os  
  
load_dotenv() # loads .env variables into environment  
api_key = os.getenv("OPENAI_API_KEY")  
client = OpenAI(api_key=api_key) # secret never in code
```



Install with `pip install python-dotenv` or `uv add python-dotenv`. Each teammate creates their own local .env file.



AI Coach John



FOLLOW

Repost



.gitignore: Protecting Yourself

The file that makes sure secrets never get committed

What it does: Tells Git which files and folders to never track or commit, which is critical for keeping `.env` files and virtual environments out of your repo.

.gitignore

```
.env
.venv/
venv/
__pycache__/
*.pyc
.DS_Store
*.log
.ipynb_checkpoints/
```

! Add `.gitignore` on day one before your first commit. Removing secrets from Git history after committing is difficult.

! If you accidentally commit a real key, treat it as compromised: rotate or revoke it immediately at the provider.





TEAM COLLABORATION

.env.example: Sharing Structure

How teammates know what keys they need, safely

What it does: A template file showing required environment variables with placeholder values, committed to Git (unlike the real `.env`).

`.env.example` (committed to Git)

```
OPENAI_API_KEY=your-openai-key-here
ANTHROPIC_API_KEY=your-anthropic-key-here
HUGGINGFACE_TOKEN=your-huggingface-token-here
```

bash - teammate onboarding step

```
cp .env.example .env
# then edit .env with real local values
```



This is the industry standard pattern in AI and software teams. Always include a `.env.example` template so developers never have to guess what variables are needed.



AI Coach John



FOLLOW

 Repost



Full Project Structure Breakdown

What a clean AI project actually looks like

```
my-ai-project/
├── .venv/                                # virtual env (gitignored)
├── .env                                  # real secrets (gitignored)
├── .env.example                          # template env vars (committed)
├── .gitignore
├── README.md                            # setup instructions
├── pyproject.toml                       # project metadata + deps
├── uv.lock                              # locked exact dependency versions
├── requirements.txt                     # (if NOT using uv/pyproject)
├── config.yaml                          # non-secret tunable settings
├── src/my_project/                      # organized source code
│   ├── __init__.py, main.py, config.py
│   ├── data/                            # sample or local data
│   ├── notebooks/                       # exploratory Jupyter notebooks
│   └── tests/test_main.py
```



This structure cleanly separates concerns: secrets stay isolated in `.env`, dependencies are locked, and code is neatly modularized under `src/`.





Config Breakdown: **pyproject.toml**

The modern, all-in-one project config file

What it does: Replaces scattered setup files (`setup.py` , `requirements.txt`) with one standard file describing metadata and dependencies. Updated automatically by `uv add` .

pyproject.toml

```
[project]
name = "my-ai-project"
version = "0.1.0"
description = "A RAG pipeline built with LangGraph"
requires-python = ">=3.10"
dependencies = [
    "langchain>=0.2.11",
    "langgraph>=0.1.0",
    "transformers>=4.44.0",
    "python-dotenv>=1.0.1",
]

[dependency-groups]
dev = ["pytest>=8.0.0", "black>=24.0.0"]
```



[project] holds core packages while **[dependency-groups]** isolates dev tools (installed via `uv add --dev`).





App & Model Config: **config.yaml**

Separating tunable settings from secrets: the part people forget

Holds non-secret settings (model choice, temperature, RAG chunking, paths) separately from `.env` for clean git versioning.

config.yaml

```
# Model settings
llm_model: "mistralai/Mistral-7B-Instruct-v0.2"
embedding_model: "BAAI/bge-small-en-v1.5"
max_tokens: 1024
temperature: 0.2

# Retrieval and RAG settings
chunk_size: 500
chunk_overlap: 50
top_k_retrieval: 5
vector_store_path: "./data/faiss_index"

# Runtime settings
environment: "development"
log_level: "INFO"
```

Python loading config.yaml

```
import yaml
with open("config.yaml") as f:
    config = yaml.safe_load(f)
max_tokens = config["max_tokens"]
```



Rule of Thumb: Secrets (API keys) go to `.env`. Tunable parameters (temperature , models) go to `config.yaml`.



Full Setup Walkthrough (uv workflow)

From empty folder to running project, step by step

bash - complete start to finish

```
# 1. Install uv once on your machine
curl -Lsf https://astral.sh/uv/install.sh | sh

# 2. Scaffold project (creates pyproject.toml automatically)
uv init my-ai-project && cd my-ai-project

# 3. Add deps (creates .venv + uv.lock automatically)
uv add langchain langgraph transformers python-dotenv pyyaml
uv add --dev pytest black

# 4. Create secret, template, and config files
touch .env .env.example config.yaml

# 5. Create .gitignore BEFORE first commit
touch .gitignore
echo -e ".env\n.venv\n\n__pycache__\n*.pyc" >> .gitignore

# 6. Initialize git and commit
git init && git add . && git commit -m "Initial setup"

# 7. Run code inside project environment
uv run main.py
```



No manual activation needed: `uv add` and `uv run` handle the environment automatically. Creating `.gitignore` before `git add .` keeps secrets out of commit history.



AI Coach John



FOLLOW

Repost

! PITFALLS & TRAPS

Common Mistakes & Security Pitfalls

What trips up almost every beginner at least once

- ✗ **Committing `.env` before setting `.gitignore`** : always configure `.gitignore` first.
- ✗ **Hardcoding keys "for testing" then forgetting**: use `.env` from day one.
- ✗ **Sharing real `.env` over Slack or email**: use `.env.example` ; never casually transmit secrets.
- ✗ **Installing packages globally**: causes severe version conflicts across projects.
- ✗ **Mixing `uv add` and `uv pip install`** : pick one mode to prevent drift between lockfile and environment.
- ✗ **Not pinning dependency versions**: use `uv.lock` or pinned requirements to prevent upstream breakage.
- ✗ **Forgetting to revoke leaked keys**: deleting code does not invalidate keys; revoke at provider immediately.



AI Coach John



FOLLOW

Repost



FINAL VERIFICATION

Quick Reference Checklist

Before your first commit, confirm all of these



Project environment created using `uv init + uv add` ,or `venv + pip` .



Dependencies explicitly tracked via `pyproject.toml + uv.lock` ,or `requirements.txt` .



`.env` created with real secrets, and `.env.example` populated with safe placeholders.



`.gitignore` created before the first `git add` ,including both `.env` and `.venv/` .



`config.yaml` set up with model settings (`max_tokens` , `temperature` ,model name), retrieval settings, and runtime settings separated from `.env` .



`README.md` included with clear setup instructions for anyone else cloning the repo.



You are ready to build! A rock-solid dev environment makes your AI applications secure, reproducible, and production-ready from day one.



AI Coach John



FOLLOW

Repost